

Usage

How to run this project, from a fresh clone to a submission.

Contents

1. [Setup](#)
 2. [Reproduce the submission](#)
 3. [Data pipeline](#)
 4. [Stage 1 — VAE](#)
 5. [Stage 2 — conditional latent diffusion](#)
 6. [Stage 3 — MIC potency scorer](#)
 7. [Generate](#)
-

1. Setup

```
uv sync
```

Everything runs from the repository root. `generate.py` resolves checkpoints/release/ and data/antibacterial.fasta relative to the current directory, which is how the validator invokes it.

2. Reproduce the submission

For inference: the released checkpoints ship in the repository.

```
uv sync
uv run generate
```

Writes generate/library.fasta (50,000 sequences) and generate/top.fasta (top 100 sequences). Every argument is at default value and the default seed is **42**, so repeated runs are identical.

3. Data pipeline

The model's inputs are committed under data/processed-data/, so this section is not needed for training or inference. It is here to process those inputs from the raw sources.

Stage 1 — DBAASP (scripts/dbaasp/, 01-05)

- 01 pages saves each raw page untouched.
- 02 flattens those into a peptide list CSV whose *string* dbaaspId column is what 03 uses to fetch peptide details.
- 04 explodes the details into activity and hemolysis frames.
- 05 standardises MIC units and filters to a medium (MHB by default).

01 and 03 are the web scraping steps and are not run here. Starting from the archives:

```
unzip -o data/raw-data/dbaasp/list_pages.zip -d data/raw-data/dbaasp/
unzip -o data/raw-data/dbaasp/details.zip -d data/raw-data/dbaasp/ # 50.9 MB
↪ MB

uv run python scripts/dbaasp/02-build-peptides-csv.py \
  --list-pages-dir data/raw-data/dbaasp/list_pages \
  --out data/raw-data/dbaasp/dbaasp-peptides-compiled.csv

uv run python scripts/dbaasp/04-build-peptide-activity-hc-df.py \
  --details_dir data/raw-data/dbaasp/details \
  --out_activity_csv data/raw-data/dbaasp/dbaasp-activity.csv \
  --out_hc_csv data/raw-data/dbaasp/dbaasp-hc.csv

uv run python scripts/dbaasp/05-standardize-mic.py
```

Running the above commands upto 04 reproduces the DBAASP activity dataframe exactly: 39 columns; 193,763 rows; 25,069 unique peptides. 05's output is the identical to data/processed-data/dbaasp-std/filtered-dbaasp.csv in the repository (17 columns; 10,663 rows).

Stage 2 — clean and combine

```
uv run python scripts/clean_and_combine/prepare-finetune.py \
  --dbaasp data/raw-data/dbaasp/dbaasp-activity.csv \
  --grampa data/raw-data/grampa/grampa.csv \
  --dramp <unzipped-dramp-dir> \
  --out data/processed-data/clean_and_combine/finetune.csv \
  --fasta data/processed-data/clean_and_combine/finetune.fasta

uv run python scripts/clean_and_combine/prepare-pretrain.py \
  --marlys <MLAMP_db.json> \
  --out data/processed-data/clean_and_combine/pretrain.csv \
  --fasta data/processed-data/clean_and_combine/pretrain.fasta
```

prepare-finetune.py applies the constraint filters (monomer, canonical residues, 8-50 aa, no terminal modifications, no intra-chain bonds). prepare-pretrain.py applies only length and disulfide filters, because MarlysAMP lacks the modification annotations.

This stage cannot run from a clone as-is, and is the one part of this document not executed. GRAMPA and the MarlysAMP JSON are not in the repository (only the DBAASP archives and data/raw-data/dramp.zip ship). Both sources are public — re-download GRAMPA from [zswitten/Antimicrobial-Peptides](https://zswitten.github.io/Antimicrobial-Peptides) (data/grampa.csv) and MarlysAMP from [Mendeley 10.17632/w4hb5grjwb.3](https://www.mendeley.com/datasets/w4hb5grjwb/3), then unzip dramp.zip. **This step's outputs are committed**, so stage 3 and everything after it run without this step.

Stage 3 — cluster and segregate

CD-HIT is run **externally** — see DATA.md for the commands — it is not a Python dependency, and its outputs are saved under data/processed-data/cluster_and_seggregate/cd-hit-output/.

```
uv run python scripts/cluster_and_seggregate/cdhit-split.py \
  --clstr data/processed-data/cluster_and_seggregate/cd-hit-
    ↳ output/finetune50_output.clstr
    ↳ \
  --reference data/processed-data/clean_and_combine/finetune.csv \
  --out data/processed-
    ↳ data/cluster_and_seggregate/seggregate/finetune_seggregated.csv

uv run python scripts/cluster_and_seggregate/filter-pretrain.py \
  --cluster data/processed-data/cluster_and_seggregate/cd-hit-
    ↳ output/pretrain50_output.clstr
    ↳ \
  --pretrain data/processed-data/clean_and_combine/pretrain.csv \
  --finetune data/processed-
    ↳ data/cluster_and_seggregate/seggregate/finetune_seggregated.csv
    ↳ \
  --out data/processed-
    ↳ data/cluster_and_seggregate/seggregate/pretrain_seggregated.csv
```

cdhit-split.py assigns a homology-aware 90/5/5 split at the cluster level (--val-frac and --test-frac, both 0.05). filter-pretrain.py then drops pretrain peptides whose cluster overlaps the finetune val/test splits.

Pretraining on a sequence homologous to a finetune test peptide means leakage. Any new data source must go through stage 3 before it is trained on.

Model inputs

data/processed/ and data/ld-processed/ are gitignored; rebuild them from the committed data/processed-data/:

```
uv run python -m src.data.clean_corpus      # output:
  ↳ data/processed/{pretrain,finetune}_clean.csv
uv run python -m src.data.build_ld_labels   # output:
  ↳ data/ld-processed/{pretraining, finetuning}.csv
```

- clean_corpus deliberately drops every column but id / sequence (/ split), so no biophysical signal can leak into the property-agnostic VAE.
- build_ld_labels prepares data for the latent-diffusion step, annotating peptide sequence with charge, gravy, hydrophobic_moment and length.

4. Stage 1 — VAE

Sequence to 64-d latent, deliberately property-agnostic. Two training phases:

```
uv run python -m src.train.pretrain \
    --train-config config/experiments/exp_dz64_fb3p0_lip0.yaml \
    --model-config config/experiments/exp_dz64_fb3p0_lip0_model.yaml
uv run python -m src.train.finetune \
    --train-config config/experiments/finetune_fb3p0_lip0p1_v1.yaml
```

Both also take `--data-config` (default `config/data.yaml`), `--model-config` (default `config/model.yaml`) and `--seed`. Checkpoints go to the `checkpoint_dir / run_name` named in the train config.

```
uv run python -m src.eval.freeze_report \
    --checkpoint checkpoints/release/finetune_fb3p0_lip0p1_v1_best.pt --split
    ↪ val
```

5. Stage 2 — conditional latent diffusion

A residual MLP with adaLN-Zero over the VAE latent, conditioned on length / charge / GRAVY with per-example random masking — which is what makes classifier-free guidance free.

Feasibility gates first. **Run this before any diffusion training:**

```
uv run python -m src.diffusion.preflight \
    --vae-checkpoint checkpoints/release/finetune_fb3p0_lip0p1_v1_best.pt
```

Every module in this stage resolves the stage-1 VAE from `checkpoints/release/`, and each accepts a `--vae-checkpoint` override, so nothing here needs a file copied into the gitignored `checkpoints/*.pt`. The flag is shown explicitly above but the default is now the same path.

```
uv run python -m src.diffusion.train --train-config
    ↪ config/train_diffusion_pretrain.yaml
uv run python -m src.diffusion.train --train-config
    ↪ config/train_diffusion_finetune.yaml

uv run python -m src.diffusion.sample \
    --checkpoint checkpoints/release/diffusion_target_v1_best.pt \
    --vae-checkpoint checkpoints/release/finetune_fb3p0_lip0p1_v1_best.pt \
    --n 64 --cfg-weight 2.0 --out-fasta samples.fasta
```

Guidance weight depends on how many properties you condition on. `cfg_weight = 2.0` was pinned on validation with all three active and is right for sampling as above. Under **charge-only** conditioning it over-extrapolates, so `generate.py` uses **1.0**.

```
uv run python -m src.eval.diffusion_report \
    --checkpoint checkpoints/release/diffusion_target_v1_best.pt \
```

```
--vae-checkpoint checkpoints/release/finetune_fb3p0_lip0p1_v1_best.pt \  
--split val --cfg-weights 1.0
```

Pass --vae-checkpoint here even though the other modules default correctly. Without the flag the run dies with `FileNotFoundError`. It also needs `data/ld-processed/` populated, so run `src.data.build_ld_labels` first.

The module defaults to `--split val`, which is safe, but its `--cfg-weights` default is a **sweep** > (1.0 1.5 2.0 3.0). Running a sweep against `--split test` is hyperparameter selection on held-out data.

Never run `src/freeze/export_frozen.py`. Stage 2 computes its own `z_stats`; a frozen VAE double-normalizes silently. `assert_vae_unfrozen` guards against it.

6. Stage 3 — MIC potency scorer

Ranks the generated library to pick the top 100. Input **sequences, not latents** — the scorer ships on descriptors only.

```
uv run python scripts/mic/01-build-mic-labels.py          # ->  
  ↳ data/processed-data/mic-labels/mic_labels.csv  
uv run python -m src.scorer.train --features f1           # cross-validation  
  ↳ (--mode cv, the default)  
uv run python -m src.scorer.train --features f1 --mode fit  
uv run python -m src.scorer.baseline                      # the no-shared-trunk  
  ↳ reference bar  
uv run python -m src.scorer.export                       # -> the release bundle
```

`--features` is required (f1 or f2). `--folds` defaults to 5 and `--seeds` to 0 1 2. Labels default to `data/processed-data/mic-labels/mic_labels.csv`, which is committed, so the label build is optional.

Reproduce the model report:

```
uv run python -m src.scorer.report --bundle checkpoints/release/scorer_v1.pt
```

This prints `expected-vs-got` and exits non-zero on drift, so it doubles as a regression test. Run it after touching featurization, the label build or the ranking.

The feature cache lives under `data/processed/scorer-cache` and is gitignored, so the first run in a fresh clone rebuilds it.

7. Generate

```
uv run generate
```

Sweeps eight charge targets (+3...+10), conditions on **charge alone** with length and GRAVY **masked**, and filters to exactly 50,000 sequences plus a ranked top 100.

Three files are written: `generate/library.fasta`, `generate/top.fasta`, and `generate/ranking.csv` (below).

Masking is deliberate. The model was trained with per-example condition masking, so a masked property is not pinned — the model supplies it.

The seven filter gates, in order

Applied per charge bucket, exactly as in `src/amp_challenge_2027/generate.py`:

#	gate	note
1	valid	8–50 residues, canonical alphabet only
2	unique	across all buckets, not merely within one
3	not an exact match to <code>data/antibacterial.fasta</code>	the only novelty rule the competition imposes
4	realized charge in (+3, +12]	realized , not requested; --min-charge exclusive, --max-charge inclusive
5	non-degenerate	one AA dominated / low-complexity decodes
6	more than --min-edits (3) edits from any reference peptide	eliminates near-duplicates
7	90% identity redundancy cap	per cluster; also beyond the rules

generate/ranking.csv — the selection and ranking record

One row per library sequence, best ensemble score first, in the same order the top-100 was drawn from. Columns:

column	meaning
rank	ensemble rank, 1 = best. Equals the row number
library_index	1-based position in <code>library.fasta</code> , i.e. the <code>>seqN id</code> — joins the two files, and makes the sort's tie-break reproducible from the CSV alone
sequence, length	
charge_requested	the +3...+10 bucket the sequence was sampled for
charge_realized	Bjellqvist charge of the decoded peptide, from the same function gate 4 uses
ensemble_score	the shipped ranking signal (breadth), at the precision the sort uses
trees_score, trees_rank	the per-species gradient-boosted trees alone
tobit_score, tobit_rank	the multi-head Tobit network alone
top100_rank	1–100 for the sequences in <code>top.fasta</code> , empty otherwise

`top100_rank` is **not** the same as `rank <= 100`: the 80% identity gate skips candidates, so the shipped top-100 is not the first 100 rows, and the gap is visible here.

Each is the same lower-confidence-bound formula applied to that family's 5 folds alone, where the ensemble uses all 10, so family scores are systematically less conservative and `ensemble_score` is *not* the mean of the two.

The resume cache will silently reproduce previously generated library

generate/.buckets/ caches per-bucket output, which makes a killed run cheap to restart. So, after any change to conditioning arguments, do:

```
rm -rf generate/.buckets
```

8. How the top-100 is selected

Ranking happens after the library is already final. It never adds or removes a sequence — the seven gates decided membership, and all this stage does is put the 50,000 survivors in an order and then take the best hundred that clear one extra novelty bar.

The scorer behind it is not a single model but **ten**: five cluster-disjoint cross-validation folds of the per-species gradient-boosted trees, and five folds of the multi-head Tobit network, each predicting potency against all six organisms. Their raw outputs are not comparable — one is a boosted tree’s conditional mean, the other a Tobit location parameter under a clamped sigma — so before anything is combined, each member’s predictions are converted to **normalized ranks** within the pool being scored. Everything downstream lives in that shared rank space, which is what lets two differently-scaled families be averaged at all without calibrating either.

This is also why the whole library is scored in a single call. Ranks are pool-relative, so scoring a subset would produce different numbers for the same peptide; and the descriptor featurization is an uncached per-sequence loop that dominates the runtime, so a second pass would cost minutes and buy nothing.

From the resulting (10, 50000, 6) array of ranks, one number per sequence is distilled in three moves (src/scorer/predict.py::_fuse):

```
mu          = ranks.mean(axis=0)                # consensus, per species
sigma       = ranks.std(axis=0)                  # how much the ten members disagree
lcb         = mu - sigma                         # lower confidence bound
breadth     = lcb.mean(axis=1) - lcb.std(axis=1) # across species, penalising unevenness
```

Both subtractions are deliberate and neither is tuned. Subtracting sigma guards against the winner’s curse: picking 100 from 50,000 is a maximum over many noisy estimates, which selects hard for candidates whose error happened to land high, so a candidate the members disagree about is demoted rather than rewarded for its luckiest fold. Subtracting the across-species spread encodes what “broad-spectrum” actually means — uniformly active, not high on average — so a peptide that is potent against two organisms and dead against four cannot win on its mean. The result is the breadth value reported as `ensemble_score`.

Sorting on it is done deliberately defensively: scores are rounded to six decimals *before* an explicitly stable sort, which turns last-bit floating-point noise into exact ties and lets those ties fall back to library order. That is what makes the ranking, and therefore the shipped files, reproducible run to run.

The top-100 is then not simply the first hundred rows. The competition holds the top list to a stricter novelty standard than the library: no entry may exceed 80% Levenshtein

identity with any sequence in `data/antibacterial.fasta`. So the ranked list is walked from the top, each candidate is compared against all 39,448 reference peptides, and it is kept only if its maximum similarity is at or below 80%; the walk stops as soon as a hundred have been kept. Screening in rank order like this, rather than taking the first hundred and then filtering, is what stops rejections from punching holes in the final list. If fewer than a hundred were ever to clear the bar the run aborts rather than shipping a short file. `top100_rank` in `generate/ranking.csv` records the position in that surviving list, which is why it is not interchangeable with `rank`.

Finally, the two per-family columns in that CSV are **diagnostics produced alongside the ensemble, never inputs to it**. Each is the same fusion applied to one family's five folds on its own, and the ensemble cannot be rebuilt from the pair — fusing ten members at once is not a function of fusing two groups of five, because neither a standard deviation nor a rank-of-a-mean decomposes that way. Averaging the two family ranks reproduces just 16 of the 50,000 ensemble ranks. Their value is showing that the ensemble is not a rubber stamp for either parent: selecting on the trees alone would have shipped a top-100 overlapping the real one by only 21 of 100, and on the Tobit network alone by 12 of 100. Inside the shipped list the trees' own ranks span 8-841 and the Tobit network's span 18-2,536, so nothing is there because one family loved it — candidates earn their place by being ranked well by both while the members agree, which is precisely what the confidence bound rewards.

Read the family scores as orderings within their own column, never as magnitudes comparable across columns. Each uses a population standard deviation over five folds where the ensemble uses ten, so family scores are systematically less conservative, and `ensemble_score` is not the mean of the two.